Project Report

on

Manufacturing System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Bhaskar Sharma	EN22CS301261
2.	Chitrak Jiyal	EN22CS301303
3.	Darshana Matade	EN22CS301309
4.	Dipanshi Yadav	EN22CS301337

Group Name: DevOps

Project Number: DO-32

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-26

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

Results & Output

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Future Scope & Enhancements

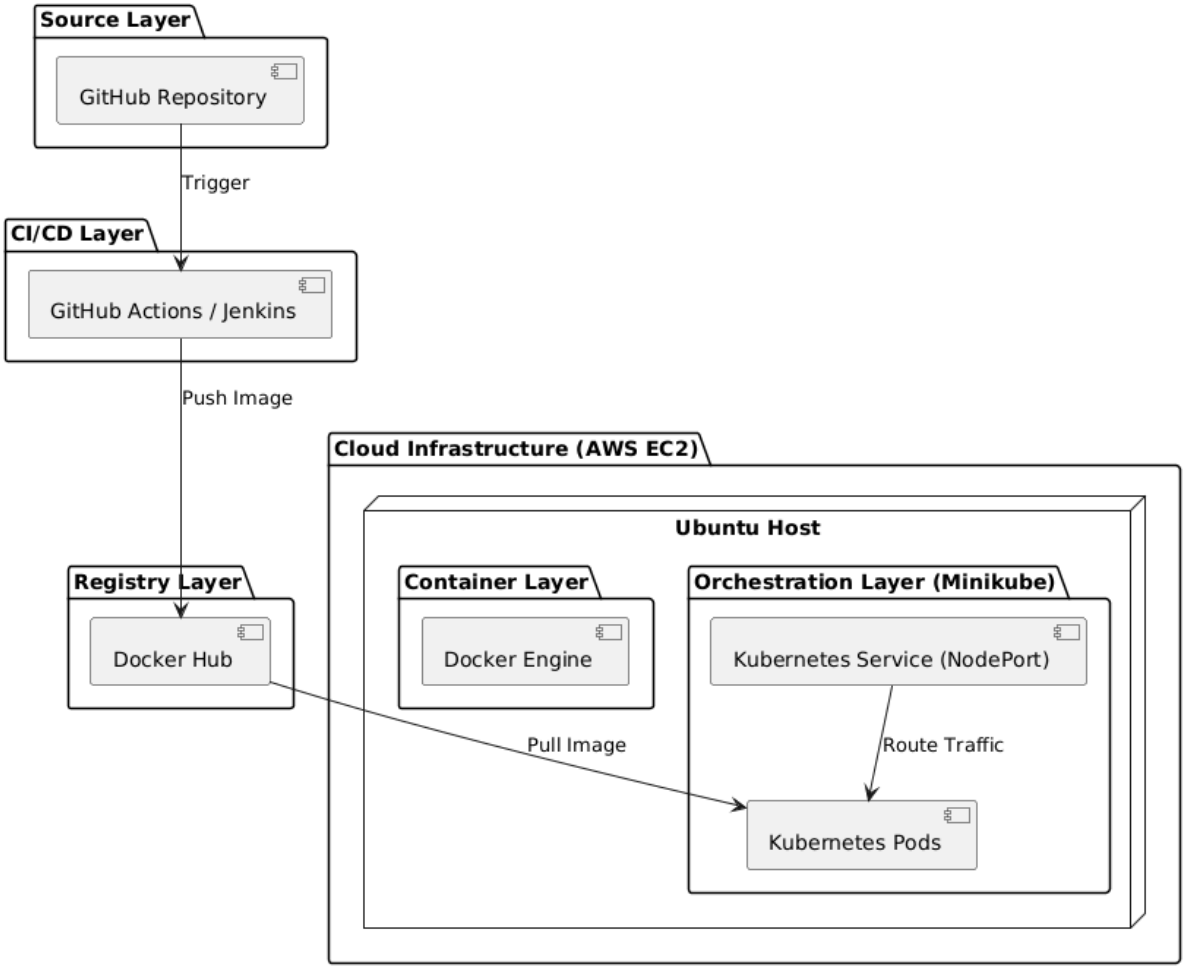
1. Problem Statement & Objectives

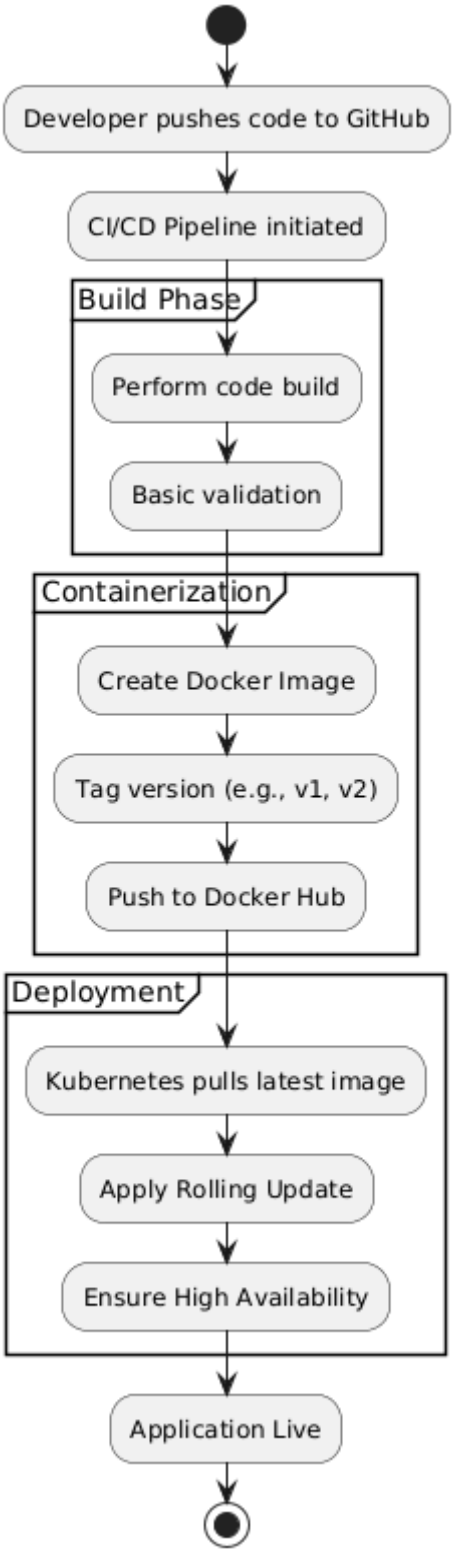
- **Problem Statement:** There is a need to automate the deployment of a Manufacturing Application using modern DevOps practices to ensure efficiency and reliability.
- **Project Objectives:**
  - Automate the build and deployment process through a centralized pipeline.
  - Ensure high availability of the application in a cloud environment.
  - Achieve zero-downtime deployments using rolling update strategies.
- **Scope of the Project:** The project covers the end-to-end setup of a CI/CD integration, overall system architecture, and a cloud-based infrastructure strategy on AWS.

2. Proposed Solution

- **Key Features:**
  - **CI/CD Integration:** Automated workflow from code push to live deployment.
  - **Containerization:** Using Docker to package the application for environment consistency.
  - **Orchestration:** Managing containers through Kubernetes to handle scaling and fault tolerance.
  - **Rolling Updates:** Ensuring the application stays live while new versions are deployed.
- **Overall Architecture / Workflow:** The architecture is organized into functional layers:
  1. **Source Layer:** Code is managed in a GitHub repository.
  2. **CI/CD Layer:** GitHub Actions or Jenkins triggers the build upon a code push.
  3. **Container & Registry Layer:** A Docker image is built and pushed to Docker Hub.
  4. **Orchestration Layer:** Kubernetes pulls the image and manages pods on an AWS EC2 instance.

5. **Service Layer:** A Kubernetes NodePort Service routes traffic to the application.



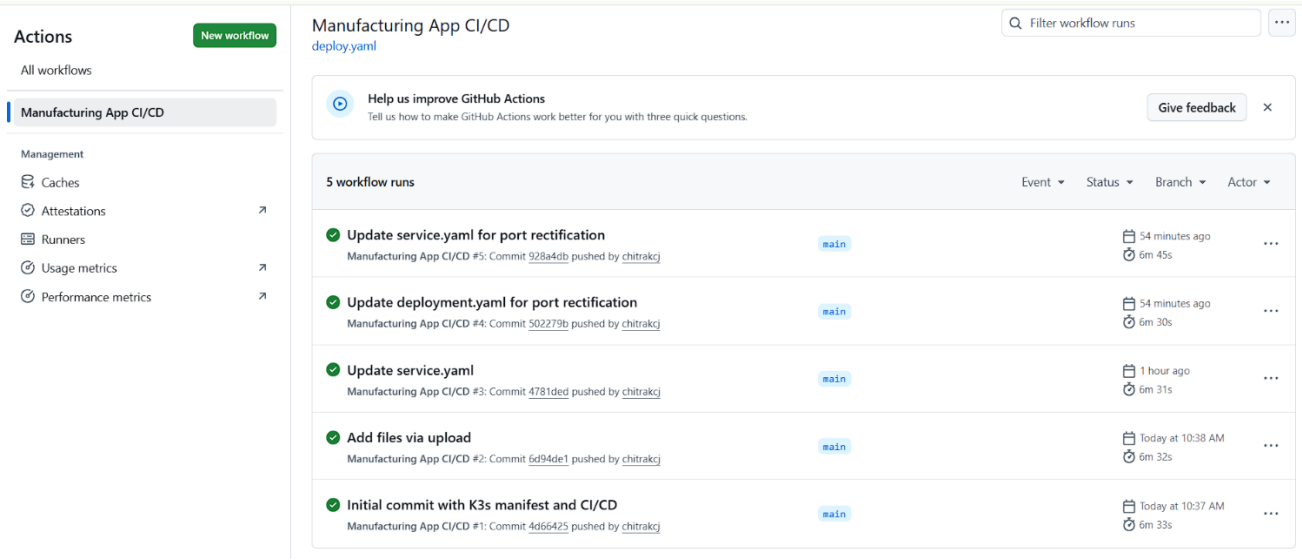


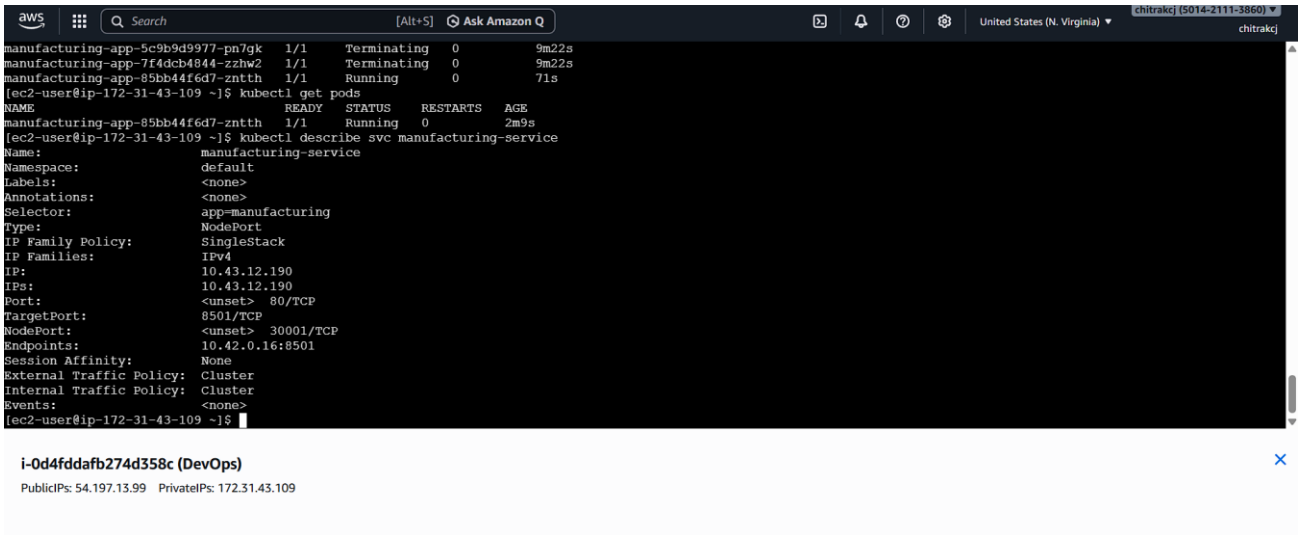
- **Tools & Technologies Used:**
  - **Cloud Host:** AWS EC2 (AWS Linux-based).
  - **CI/CD Pipeline:** GitHub Actions.

- **Containerization:** Docker Engine.
- **Orchestration:** Kubernetes (Minikube used for design).
- **Registry:** Docker Hub.

3. Results & Output

- **Screenshots / Outputs:**





- **Reports / Dashboards / Models:** The data design centers on environment variables for configuration and Kubernetes Secrets for handling sensitive credentials securely.
- **Key Outcomes:**
  - Reduction in manual intervention through full deployment automation.
  - Enhanced fault tolerance, with Kubernetes automatically restarting failed pods.
  - Versioned image retention in Docker Hub, allowing for rapid rollbacks if necessary.

4. Conclusion

The DevOps HLD successfully establishes a modular and scalable deployment framework for the Manufacturing System. By integrating GitHub Actions, Docker, and Kubernetes, the system ensures that software delivery is fast, consistent, and highly available.

5. Future Scope & Enhancements

- **Scaling:** Expanding horizontal scaling by increasing Kubernetes replicas based on load.
- **Security:** Further hardening of Docker images to prevent unauthorized access.
- **Monitoring:** Implementing advanced logging and performance monitoring within the Kubernetes cluster.
- **Multi-Cloud:** Extending the infrastructure strategy to support hybrid or multi-cloud deployments.